

Buổi 1

Giới thiệu về API và Web Services

Mục tiêu & Nội dung

Định nghĩa được API, Web Services, REST

Giải thích được sự khác biệt giữa các loại API và khi nào nên dùng loại nào.

Nhận biết API phù hợp cho tình huống cụ thể

Phân loại được các hệ thống dựa trên API mà chúng sử dụng, đề xuất giải pháp API cho bài toán mới.

PHẦN 1	PHẦN 2	PHẦN 3	PHẦN 4
Khái niệm và lịch sử API • API là gì • Đặc điểm & vai trò • Mốc lịch sử 1998–2015	Phân biệt khái niệm • API vs Web Service • Microservices • Kiến trúc Client-Server	Các loại API • REST • SOAP · GraphQL · gRPC • Ma trận lựa chọn	Thực hành với Flask • Cài đặt môi trường • Bài tập thực hành • Tạo mini CRUD API

01

Khái niệm và lịch sử phát triển API

API thực chất là gì? nó xuất hiện khi nào? và tại sao nó trở thành hạ tầng không thể thiếu của phần mềm hiện đại.

- Định nghĩa được API theo nghĩa rộng và nghĩa hẹp
- Mô tả được 5 đặc điểm cốt lõi của một API hiện đại
- Lịch sử phát triển từ 1998–2015

API là gì?

API (Application Programming Interface)

là tập hợp **quy tắc + giao thức + công cụ** để **các hệ thống phần mềm** giao tiếp: một bên yêu cầu dữ liệu/chức năng, bên kia trả lời theo hợp đồng đã công bố.

API được dùng theo hai nghĩa

Nghĩa 1

API của thư viện / framework

Hàm, lớp, kiểu mà thư viện cung cấp cho lập trình viên. VD: `sorted(list)`, `Array.map()`.

Quy mô: **bên trong project**, truy cập qua `import/include`.

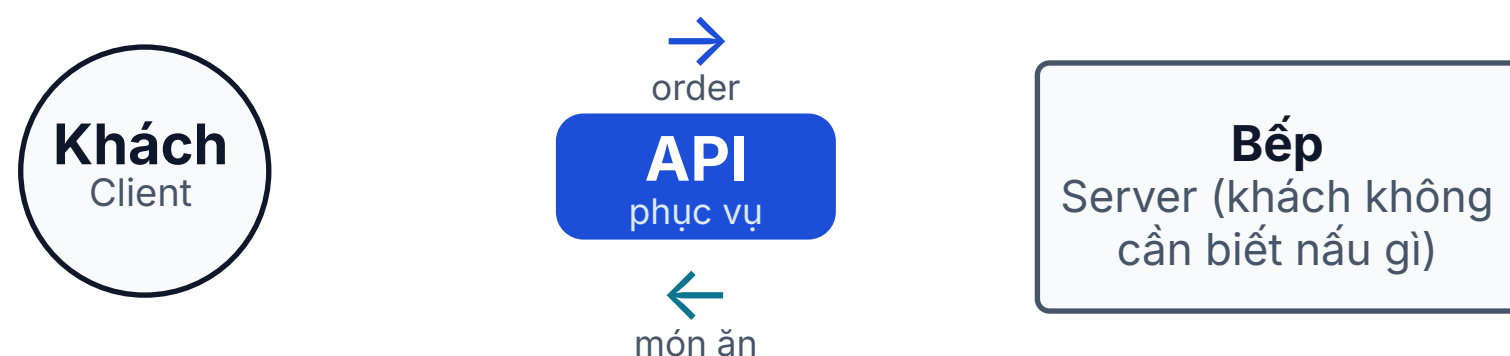
Nghĩa 2

API giao tiếp qua mạng

Endpoint + giao thức (HTTP, gRPC, GraphQL) để hai dịch vụ trao đổi. VD: GitHub REST API, Stripe Payments API.

Quy mô: **qua mạng**, truy cập qua HTTP request.

Ví dụ



Khách gọi món qua **phục vụ**, không cần biết **bếp** nấu gì, nấu thế nào.

Bồi bàn chính là *hợp đồng giao tiếp*: "có món nào, giá bao nhiêu, gọi bằng cách nào", tương tự một API endpoint.

Năm đặc điểm cốt lõi của một API hiện đại

1 Có hợp đồng rõ ràng

Contract-first

Endpoint, method, request body, response shape, error codes được **mô tả trước** khi viết code. Client có thể đọc spec mà không cần đọc code.

VD: OpenAPI 3.x, GraphQL SDL, Protobuf IDL.

2 Ẩn cài đặt

Abstraction

Client không biết, và **không nên biết**, server xử lý thế nào. Database là gì, ngôn ngữ gì, infra ra sao đều là chi tiết triển khai nội bộ.

Đổi Postgres → MongoDB không ảnh hưởng client.

3 Có thể tái sử dụng

Reusability

Một client (có thể từ team khác) gọi được nhiều lần mà không cần code phía server phải đổi. API dùng để chia sẻ chức năng chứ không copy/paste.

Stripe phục vụ hàng triệu shop từ 1 codebase.

4 Độc lập với ngôn ngữ & nền tảng

Language-agnostic

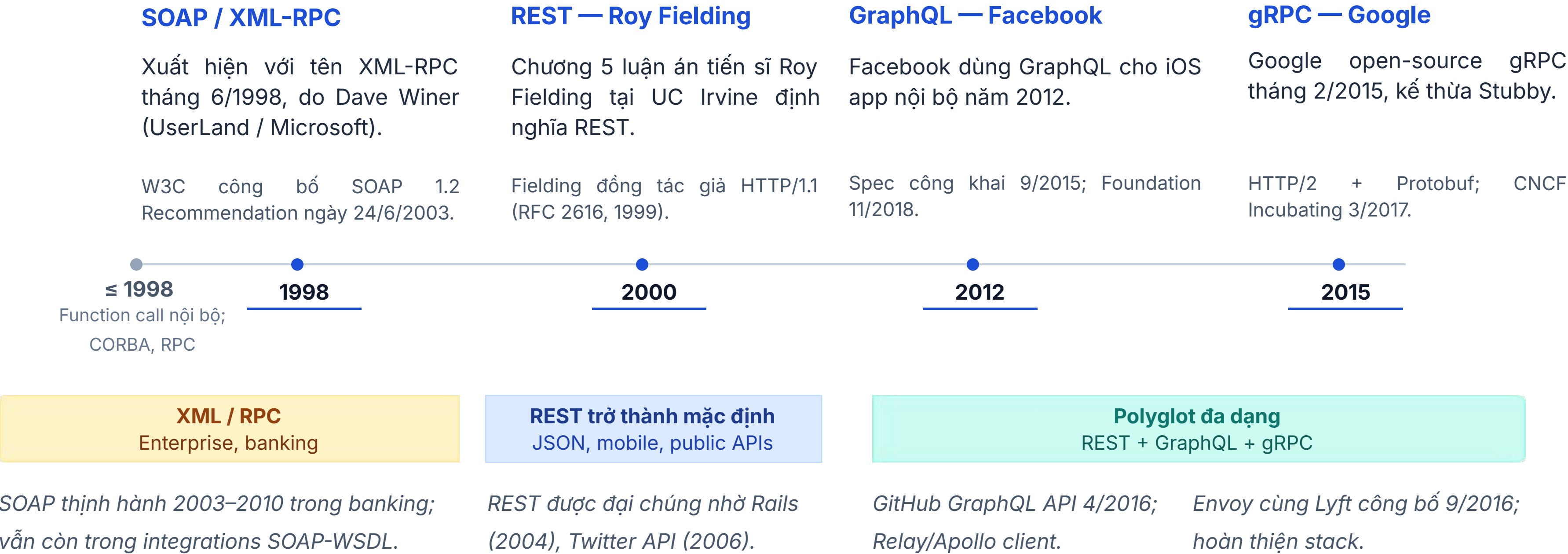
Một API HTTP/JSON có thể được gọi từ **Python, Node.js, Go, Java, PHP, .NET, mobile**. Định dạng truyền thống (JSON, XML, Protobuf) là trung gian, không ai "sở hữu" API.

5 Có thể versioning & quan sát được

Versioned & Observable

Một API chuyên nghiệp phải có **version** (v1, v2...) để tiến hóa mà không phá client cũ, kèm **logs, metrics, traces** để vận hành (nội dung buổi 9 và 10)

Lịch sử phát triển API từ 1998 – 2015



Phân biệt API, Web Service & Microservice

API — giao tiếp & hợp đồng (interface)

Web Service — API truy cập qua web (medium)

Microservice — cách triển khai (architecture)

02

API ≠ Web Service ≠ Microservice

API

hợp đồng là gì?

Hợp đồng giao tiếp giữa hai bên, có thể chạy trên bất kỳ môi trường nào.

Đặc điểm

- Là một **hợp đồng**, không phải một hệ thống
- Không gắn với công nghệ cụ thể
- Có thể là library-level *hoặc* network

Ví dụ

- `java.util.List` API
- `os/300` OS API
- GitHub REST API

Web Service

truyền qua đâu?

Là một **API có thể truy cập qua mạng** (thường là HTTP), có URL và trả response.

Đặc điểm

- Đi qua **mạng** (HTTP)
- Bắt buộc có URL/URI
- SOAP + REST là 2 kiểu chính

Ví dụ

- `api.weather.gov`
- `graph.facebook.com`

Microservice

tổ chức thế nào?

Một **kiến trúc phần mềm** chia hệ thống thành nhiều service nhỏ, độc lập.

Đặc điểm

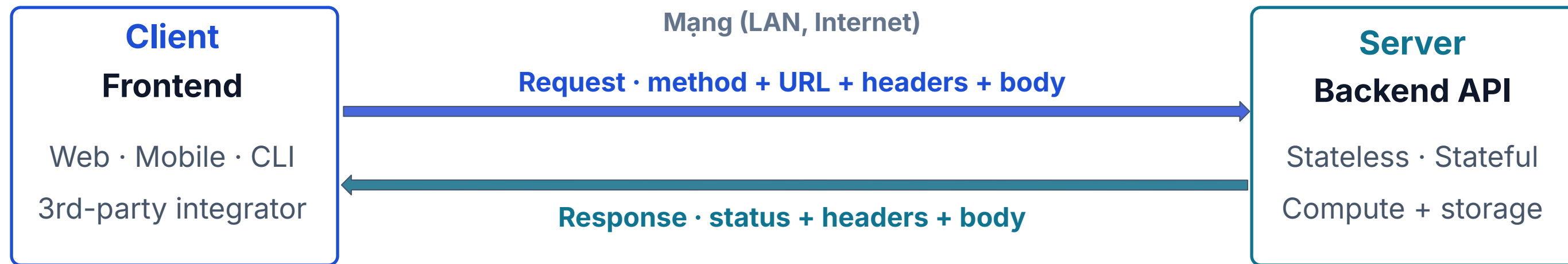
- Một dịch vụ = **một business capability**
- Giao tiếp với nhau qua API
- Độc lập deploy, scale riêng

Ví dụ

- `orders-service` viết bằng Go
- `inventory-service` viết Python

Web service chạy trên kiến trúc Client-Server

Giao tiếp qua **mạng**. Client *không* biết server làm gì, server *không* biết client là ai, chỉ quan tâm **request** và **response**.



REQUEST — client gửi

```
POST /api/v1/orders HTTP/1.1
Host: shop.example.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOi...
{"sku":"BK-001","qty":2}
```

RESPONSE — server trả về

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/v1/orders/ord_42
{
  "id":"ord_42","status":"pending"
}
```

03

Một số loại API phổ biến

REST, SOAP, GraphQL, gRPC, mỗi cái được sinh ra cho một tình huống khác nhau. Cần nhận diện, phân biệt, và chọn đúng công cụ cho đúng job.

SOAP
1998/2003

REST
2000

GraphQL
2012/2015

gRPC
2015

- Mô tả ưu/nhược của từng loại
- Đọc được một request của mỗi loại (URL, envelope, query, message)
- Biết khi nào nên dùng cái nào, không phải lúc nào cũng REST

SOAP: XML envelope & WSDL

SOAP = giao thức RPC-style dùng **XML envelope**, định nghĩa message và kiểu trong **WSDL**. Đặc trưng: *strong contract, formal, doanh nghiệp nặng*.

Xuất hiện 6/1998 (XML-RPC · Dave Winer, UserLand) — W3C Recommendation ngày 24/6/2003.

Format
XML (only)

không JSON, không
protobuf

Transport
HTTP/SMTP

thường POST +
SOAPAction

Mô tả
WSDL

Web Services
Description Language

Một request SOAP trông như thế nào

POST /BankService HTTP/1.1 · Content-Type: text/xml

```
<?xmlversion="1.0"?>
<soap:Envelopexmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <GetAccountBalancexmlns="http://bank.example.com/">
      <accountId>ACC-99281</accountId>
    </GetAccountBalance>
  </soap:Body>
</soap:Envelope>
```

Điểm mạnh

- Schema **WSDL** là cam kết pháp lý
- **WS-Security** cho chữ ký XML
- Built-in **fault** model
- Reliable qua MQ/SMTP

Điểm yếu

- Nặng: payload x~**3-5 JSON**
- Phải có tool generate stub
- Browser không debug được
- Học khó, cần XML schema

Khi nào dùng SOAP

Banking/finance legacy (SWIFT messages, core banking) · **ERP integration** (SAP, Oracle) · **Government/Health** ở Mỹ, EU (HL7 v2 dùng SOAP) · **Tích hợp B2B** đã có hợp đồng dài hạn.

Với hệ thống mới 2024+: *không* chọn SOAP trừ khi bị ràng buộc.

REST: Representational State Transfer

REST **không phải là một giao thức**, đó là **phong cách kiến trúc** cho hệ phân tán. Một hệ thống "RESTful" là một hệ thống *tuân thủ 6 ràng buộc* mà Fielding rút ra từ cách Web hoạt động.

Roy T. Fielding (2000), "Architectural Styles and the Design of Network-based Software Architectures", UC Irvine — Chapter 5.

Sáu ràng buộc của một hệ REST

1 Client-Server

Tách UI (client) khỏi data store (server), phát triển độc lập.

2 Stateless

Mỗi request mang đủ thông tin, server không nhớ state giữa các request.

3 Cacheable

Response ghi rõ có cache được không; client/CDN có thể tái sử dụng.

4 Uniform Interface

Cùng cách gọi ở mọi resource, đây là điều làm REST khác biệt RPC.

5 Layered System

Client không biết qua bao nhiêu proxy / CDN / server trung gian.

6 Code-On-Demand

Tùy chọn: server gửi code (JS) chạy ở client (ít dùng)

VD: GitHub REST API

```
$curl -s -H"Accept: application/vnd.github+json" \
"https://api.github.com/users/octocat/repos?per_page=3&sort=updated"
→ 200 OK · [{"id":1296269,"name":"hello-world"...}, ...]
```

GraphQL: client định nghĩa shape dữ liệu

Một **endpoint duy nhất** (POST /graphql) cho mọi loại truy vấn. Client viết *query* chỉ rõ trường cần lấy → server trả về đúng và chỉ đúng những trường đó trong JSON.

Schema SDL là single source of truth.

GraphQL giải quyết vấn đề

REST over-fetch: GET /users/42 trả cả đồng trường không dùng

REST under-fetch: cần ≥3 round-trip lấy user+posts+comments

Một query GraphQL — GitHub API thật

```
POST /graphql · body = query string
query {
  user(login: "octocat") {
    name
    avatarUrl
    repositories(first: 3) {
      nodes { name }
    }
  }
}
```

→ response

```
{
  "data": {
    "user": {
      "name": "The Octocat",
      "avatarUrl": "https:// ...",
      "repositories": {
        "nodes": [{"name": "hello-w"}]
      }
    }
  }
}
```

Đổi lại

Được

Một round-trip cho dữ liệu phức tạp · Client tự điều khiển shape payload · Schema introspection tự sinh docs

Mất

HTTP caching không dùng được · File upload dùng mutation riêng · **N+1 query** server-side dễ xảy ra · Cần build schema nghiêm túc

gRPC: gọi function từ xa qua HTTP/2

gRPC dùng **HTTP/2 + Protobuf binary**. Client gọi hàm trên server, không phải lấy resource. Sinh ra để liên lạc microservice nội bộ, nơi tốc độ và đảm bảo contract quan trọng hơn tính dễ đọc.

4 kiểu streaming

- **Unary** — 1 req → 1 res
- **Server-stream** — 1 req → N res
- **Client-stream** — N req → 1 res
- **Bi-directional** — N ↔ N

Định nghĩa service bằng file **.proto**

books.proto · service definition

```
syntax = "proto3";
service BookService {
  rpc ListBooks(ListReq) returns(ListRes);
  rpc GetBook (IdReq) returns(Book);
}
message Book {
  string id = 1;
  string title = 2;
  string author = 3;
}
```

Tại sao công ty lớn chọn gRPC

Nhanh. Protobuf ~5–10× nhỏ hơn JSON cùng nội dung.

Schema = code. Sinh client + server stub cho 11 ngôn ngữ (Go, Java, Python, Node...)

HTTP/2 cho phép multiplexing + stream thật.

Strict contract: breaking change là compile error.

Dùng: microservice-to-microservice nội bộ · mobile ↔ backend low-latency · đa ngôn ngữ.

Tránh: API public cho browser (cần proxy) · khi đội chưa quen protobuf.

4 loại API phổ biến

So sánh về: định dạng dữ liệu, cấu trúc giao tiếp, mô hình kết nối.

SOAP XML-RPC 1998 · W3C 2003	REST Fielding 2000	GraphQL Facebook 2012 · open 2015	gRPC Google 2015
Tóm tắt XML envelope + WSDL mô tả. RPC-style gọi method. Gắn chặt schema và error handling.	Tóm tắt Resource + HTTP verbs. JSON là phổ biến nhất, XML cũng có. Stateless, cacheable.	Tóm tắt Một endpoint, client <i>tự định nghĩa</i> shape response. Schema SDL là single source of truth.	Tóm tắt HTTP/2 + Protobuf binary. RPC (gọi function từ xa), 4 kiểu streaming.
format XML only	format JSON · XML · YAML	format JSON (query ⇒ data)	format Protobuf (binary)
Đại diện Banking legacy (SWIFT, Core banking), SAP, gov	Đại diện GitHub API, Stripe API, Twitter API v2, OpenWeather	Đại diện GitHub GraphQL, Shopify, Atlassian, Apollo federation	Đại diện Service-to-service trong Google, Netflix, Square
Ưu — Nhược Strong contract, WS-Security Verbose, nặng, chậm adopt	Ưu — Nhược Đơn giản, dễ debug bằng curl Over-fetch, under-fetch cố hữu	Ưu — Nhược Không over/under-fetch Caching, file upload phức tạp	Ưu — Nhược Nhanh, schema strict Không đọc được = khó debug

Khi nào chọn cái nào?

"REST cho mọi thứ" là sai.

Câu hỏi 1

Đối tượng gọi là ai?

Browser / mobile / 3rd-party public? → **REST** (JSON) · hoặc **GraphQL** (nếu client rất đa dạng)
Service-to-service nội bộ (đa ngôn ngữ)? → **gRPC**
Hệ thống enterprise đã có WSDL? → **SOAP**

Câu hỏi 2

Client cần dữ liệu gì?

Mỗi client cần một phần khác nhau của cùng resource?
→ **GraphQL** (hơn REST vì tránh over/under-fetch) · nếu không cần tùy biến thì REST là đủ dùng.
Cần upload file lớn, stream real-time?
→ **REST + chunked** phổ biến; **gRPC streaming** nếu cần 2 chiều + nội bộ.

Câu hỏi 3

Cần hợp đồng?

Có nhiều team phụ thuộc? Cập nhật có lỗi với tỉ lệ thấp?
→ **Protobuf + gRPC**: breaking change là compile error, không phải production outage.
OpenAPI / JSON-Schema cũng enforce được contract, nhưng yếu hơn ở wire-level type safety.

Câu hỏi 4

Tình trạng dự án hiện tại?

Phải tương thích với SOAP / WSDL có sẵn?
→ Tái sử dụng SOAP, không nên viết lại bằng REST trừ khi có migrate plan.
Một team, một ngôn ngữ, internal only?
→ **REST + JSON** đơn giản là đủ — không over-engineer.

Trong công nghiệp từ 2024: REST + JSON cho public API · GraphQL cho client đa dạng · gRPC cho nội bộ · SOAP chỉ khi bắt buộc.

Thực hành với Flask: REST + JSON

B1 · Endpoint trả JSON

B3 · Đọc JSON từ request body

B5 · Status codes + lỗi validation

B2 · Multiple HTTP methods

B4 · Path params + query string

B6 · CRUD in-memory (tích hợp B1→B5)

04

Công cụ sử dụng

Python 3.10+

Flask 3.x

curl (để test)

Chuẩn bị môi trường Flask

Yêu cầu

Python 3.10 trở lên · Flask 3.x (Flask 3 phát hành 9/2023) · curl (mọi macOS/Linux, Windows 10+)

Vì sao Flask cho buổi 1

Không sinh boilerplate · chỉ phơi bày request/response JSON · đủ nhỏ để thấy "API = hàm + URL", đủ thật để chạy production.

1 Tạo virtual environment + cài Flask venv tránh xung đột dependency giữa nhiều project.

```
$python3 -m venv .venv
$source .venv/bin/activate # Windows: .venv\Scripts\activate
$pip install "Flask ≥ 3.0"
$flask --version
→ Python 3.11.x · Flask 3.x.x # check version
```

2 Tạo cấu trúc thư mục cho project

```
api_session_01/
├── .venv/ # môi trường ảo (đã tạo ở bước 1)
├── requirements.txt # ghi: Flask ≥ 3.0
└── app.py # file chính – buổi 1 chỉ cần file này
```

Tip · Trong VS Code: chọn interpreter là `.venv/bin/python`.

Endpoint đầu tiên: trả JSON, không trả HTML

app.py

```
# app.py – Bài 1: Hello API
from flask import Flask
app = Flask(__name__)
@app.route("/")
def index():
    return {"message": "Hello, API!"}
if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=True)
```

execute + test

```
# Bước 1 – Chạy server
$ python app.py

* Running on http://127.0.0.1:5000
```

```
# Bước 2 – Mở terminal khác, gọi
curl -i http://127.0.0.1:5000/
HTTP/1.1 200 OK
Content-Type: application/json
{"message": "Hello, API!"}
```

1 Flask() — đối tượng app

Một instance duy nhất đại diện cho toàn bộ ứng dụng web. Trong app này có: danh sách routes, config, các extension.

2 @app.route("/") — decorator

Đăng ký hàm Python với URL. GET "/" — root path — trở đến hàm `index()`. Flask đọc URL và gọi hàm tương ứng.

3 return dict — auto-JSON

Flask 3.x **tự động** chuyển dict → JSON response + Content-Type đúng. Không cần `jsonify()`.

Vì sao JSON chứ không phải HTML? API ≠ trang web.

Trả HTML thì browser hiển thị; trả JSON thì *bất kỳ client nào* (cURL, mobile app, server khác, IoT) cũng parse được. **Không** được nhầm API với website.

Nhiều routes & HTTP methods trong một file

app.py · mở rộng từ bài 1

```
from flask import Flask, jsonify, request
app = Flask(__name__)
# GET /health — kiểm tra server còn sống
@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "ok"}), 200
# POST /echo — trả lại cái client gửi
@app.route("/echo", methods=["POST"])
def echo():
    data = request.get_json(silent=True) or {}
    return jsonify({"you_sent": data}), 200
if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=True)
```

Test từng endpoint

GET /health

```
curl -i http://127.0.0.1:5000/health
```

```
HTTP/1.1 200 OK
```

```
{"status": "ok"}
```

POST /echo

```
curl -X POST http://127.0.0.1:5000/echo \
-H "Content-Type: application/json" \
-d '{"name": "An", "age": 21}'

{"you_sent": {"name": "An", "age": 21}}
```

Cần chú ý các thông tin

- **methods=...** mặc định chỉ GET — phải liệt kê method muốn cho phép. POST chỉ nhận body khi header Content-Type đúng.
- **jsonify()** trả tuple (body, status) — Flask tự đặt Content-Type = application/json.

- **request.get_json(silent=True)** trả None nếu body không phải JSON. Truyền False nếu muốn Flask raise 400.

- **Đặt debug=True** chỉ khi dev local — production *không* bật vì lộ traceback & cho phép chạy code từ xa.

Đọc JSON từ client, validate, tạo resource trả về 201 Created

Bài toán: tạo student, validate, trả về 201 (RFC 9110)

```
from flask import Flask, jsonify, request
from uuid import uuid4
app = Flask(__name__)
STUDENTS = []
@app.route("/students", methods=["POST"])
def create_student():
    body = request.get_json(silent=True) or {}
    name = body.get("name")
    if not name:
        return jsonify({"error": "name là bắt buộc"}), 400
    student = {
        "id": str(uuid4()),
        "name": name,
        "gpa": body.get("gpa", 0.0),
    }
    STUDENTS.append(student)
    return {"id": ..., "name": ...}, 201
```

Test case

Hợp lệ → 201 Created

```
$ curl -X POST localhost:5000/students \
-H "Content-Type: application/json" \
-d '{"name": "An", "gpa": 3.4}'
HTTP/1.1 201 Created
Location: /students/7f3...
{"id": "7f3...", "name": "An", "gpa": 3.4}
```

Thiếu "name" → 400

```
$ curl -X POST localhost:5000/students \
-H "Content-Type: application/json" \
-d '{"gpa": 3.4}'
HTTP/1.1 400 Bad Request
{"error": "name là bắt buộc"}
```

Một số lưu ý

201 Created khi tạo xong — RFC 9110 §15.3.2.

Response nên có header `Location` trỏ resource mới (REST convention).

uuid4() tạo id 128-bit ngẫu nhiên. Production

dùng DB sequence hoặc ULID — buổi 1 dùng uuid cho gọn.

Đừng trả stacktrace/HTML khi lỗi — trả `{"error": "..."} với status 4xx/5xx rõ ràng.`

Path params (bắt buộc) và query string (tuỳ chọn)

Path params · biến trong URL

```
# /books/<id> – id là string
@app.route("/books/<book_id>", methods=["GET"])
def get_book(book_id):
    book = find_by_id(book_id)
    if book is None:
        return jsonify({"error": "not found"}), 404
    return jsonify(book), 200
# Ép kiểu int ngay từ URL
@app.route("/items/<int:item_id>")
def get_item(item_id): # int sẵn
    return jsonify({"id": item_id}), 200
```

Query string · bộ lọc / phân trang

```
# /books?limit=10&offset=0&q=python
@app.route("/books", methods=["GET"])
def list_books():
    limit=int(request.args.get("limit", 20))
    q=request.args.get("q","").strip().lower()
    items=[b for b in BOOKS if q in b["t"].lower()]
    return jsonify({"items": items}), 200
# offset, dùng cho buổi 5
# offset=int(request.args.get("offset",0))
```

Phân biệt rõ khi nào dùng cái nào

Path param dùng khi trỏ đến **một resource cụ thể** —
/users/42, /orders/ord_921. Không thể bỏ qua.

Query string dùng khi **lọc/sắp xếp/phân trang** —
?limit=20&sort=-updated.

3 URL thật trông thế này

GET /books/abc-1234 → find by id "abc-1234"

GET /books?limit=5&q=python → 5 cuốn có "python" trong title

Đừng: ~~GET /get-book/abc-1234~~ · động từ trong URL = anti-pattern REST

Một API trả 200 cho mọi thứ = API tệ

Mô hình phân loại trước

2xx

Thành công

3xx

Redirect

4xx

Lỗi client

5xx

Lỗi server

1xx

Informational

Code minh họa — DELETE có đầy đủ status

```
from flask import Flask, jsonify
ORDERS = {} # giả lập DB
# DELETE /orders/<id>
@app.route("/orders/<id>", methods=["DELETE"])
def delete_order(order_id):
    order = ORDERS.get(order_id)
    # 404 — không tìm thấy
    if order is None:
        return {"error": "not found"}, 404
    # 409 — business rule
    if order["status"] in ("shipped", "delivered"):
        return {"error": "cannot delete"}, 409
    ORDERS.pop(order_id, None)
    # 204 — success, no body
    return "", 204
```

Bảng tra cứu nhanh

200 OK · GET/PUT/PATCH thành công
201 Created · POST tạo mới
204 No Content · DELETE thành công

400 Bad Request · JSON lỗi
404 Not Found · Không có
409 Conflict · State xung đột
422 Unprocessable · Logic sai
429 Too Many · Rate limit

500 Internal · Lỗi không rõ
503 Unavailable · Bảo trì

CRUD cho resource **books**

app.py · 5 method chia làm 2 cụm

```
from flask import Flask, jsonify, request
app = Flask(__name__)
_next = 1
BOOKS = [{"id":1,"title":"Clean Code","author":"R. Martin"}]
def find(bid):
    return next((b for b in BOOKS if b["id"]==bid), None)
# LIST - GET /books
@app.route("/books", methods=["GET"])
def list_books():
    n = int(request.args.get("limit", 100))
    return jsonify(BOOKS[:n]), 200
# DETAIL - GET /books/<int:bid>
@app.route("/books/<int:bid>", methods=["GET"])
def get_book(bid):
    book = find(bid)
    if not book: return {"error":"not found"}, 404
    return jsonify(book), 200
# CREATE - POST /books
@app.route("/books", methods=["POST"])
def create_book():
    global _next
    body = request.get_json(silent=True) or {}
    t, a = body.get("title"), body.get("author")
    if not t or not a:
        return {"error":"need title+author"}, 400
    book = {"id":_next, "title":t, "author":a}
    _next += 1; BOOKS.append(book)
    return jsonify(book), 201, {"Location":f"/books/{book['id']}"}
# UPDATE - PUT, DELETE - DELETE (xem bên phải)
@app.route("/books/<int:bid>", methods=["PUT", "DELETE"])
def modify_book(bid):
    book = find(bid)
    if not book: return {"error":"not found"}, 404
    if request.method == "PUT":
        book.update(request.get_json(silent=True) or {})
        return jsonify(book), 200
    BOOKS.remove(book)
    return "", 204
```

Schema (mỗi book)

```
{"id": int, "title": str, "author": str }
```

Endpoint ↔ status

200 · list / detail / update

201 · create (+ Location header)

204 · delete (no body)

400 · thiếu title hoặc author

404 · id không tồn tại

Tại sao gộp PUT + DELETE?

Cùng URL pattern → dùng decorator `methods=[..., ...]`.

Trong hàm phân nhánh theo `request.method`, đoạn `if`

`request.method == "PUT"` xử lý update, phần còn lại xử lý delete.

Cách khác (tách riêng 2 hàm)

```
@app.route("/books/<int:bid>", methods=["PUT"])
```

```
def update_book(bid): ... # return 200/404
```

```
@app.route("/books/<int:bid>", methods=["DELETE"])
```

```
def delete_book(bid): ... # return 204/404
```

Dùng curl để test các endpoints

GET LIST

```
$ curl http://127.0.0.1:5000/books
```

Response 200 OK

```
[{"id":1,"title":"Clean Code"...},  
{"id":2,"title":"Pragmatic..."}]
```

GET DETAIL

```
$ curl http://127.0.0.1:5000/books/1
```

404 nếu id không có

```
HTTP/1.1 404 Not Found  
{"error":"not found"}
```

POST CREATE

```
$ curl -X POST localhost:5000/books \  
-H "Content-Type: application/json" \  
-d '{"title":"DDIA","author":"Kleppmann"}'  
Trả về 201
```

```
HTTP/1.1 201 Created  
Location: /books/3  
{"id":3,"title":"DDIA","author":"Kleppmann"}
```

PUT UPDATE

```
$ curl -X PUT localhost:5000/books/1 \  
-H "Content-Type: application/json" \  
-d '{"title":"CC 2nd ed."}'
```

Trả về 200

```
HTTP/1.1 200 OK  
{"id":1,"title":"CC 2nd ed.,"author":"R. Martin"}
```

DELETE xóa

```
$ curl -X DELETE localhost:5000/books/2  
Trả về 204, body rỗng
```

```
HTTP/1.1 204 No Content  
(không có body)
```

POST 400 BAD

```
$ curl -X POST localhost:5000/books \  
-H "Content-Type: application/json" \  
-d '{}'# body rỗng
```

Validate fail → 400

```
HTTP/1.1 400 Bad Request  
{"error":"title+author required"}
```

Các lỗi thường gặp khi mới viết API

Review lại code xem có gặp những lỗi phổ biến này không?

01 Trả 200 cho mọi thứ, kể cả lỗi
Body chứa `{"success":false}` nhưng status = 200
→ client không retry, monitoring không alert, log aggregator khó tách lỗi.

```
return {"error":...}, 200 ← sai  
return ..., 400 ← đúng
```

02 Động từ trong URL
`/getUser/42, /createOrder, /deleteAccount`
→ đây là phong cách RPC, không phải REST. HTTP method đã là "verb".

```
/getUser ← sai  
GET /users/42 ← đúng
```

03 In-everything validation hoặc không validate gì cả
kiểm tra *quá ít* (tin tưởng client) → KeyError 500, hoặc
kiểm tra *mọi thứ* (mỗi hàm validate riêng) → khó tái sử dụng.

```
# Buổi 6+ dùng pydantic  
class BookIn(BaseModel):  
    title: str = Field(min_length=1)  
    year: int = Field(ge=1900)
```

04 Trộn business logic với HTTP layer
Toàn bộ tính toán nằm trong route → unit test phải qua HTTP,
có service_layer thì test được service_layer trước khi test route.

```
# tách lớp  
def route_create(b):  
    return create_book(b)  
def create_book(b):  
    # test được
```

05 Hard-code URL/secret trong code, không log gì cả
gh repo có API key → phải đọc từ env. Không log request body có password.

Tổng kết

1 API là hợp đồng giao tiếp

Có thể là library-level (nội bộ tiến trình) hoặc network-level (qua HTTP/gRPC).

Web Service là API qua web. Microservice là cách triển khai, không phải "API".

2 Tùy vào nhu cầu sử dụng để chọn loại API

REST = mặc định public

SOAP = legacy doanh nghiệp

GraphQL = client đa dạng về data shape

gRPC = service-to-service nội bộ

3 REST = Client-Server + 5 ràng buộc

Stateless, cacheable, uniform interface là core.

Không phải API nào trả về JSON cũng có thể được gọi là RESTful — Fielding yêu cầu nhiều hơn thế.

4 Một mini CRUD Flask chạy được

5 endpoint + status codes đúng + JSON I/O.

Đây là *nền tảng* cho toàn bộ Buổi 2-13: nếu bài 6 chưa chạy, dành thêm giờ cuối tuần.

Tự kiểm tra

- Có giải thích được API, Web Services, REST khác nhau thế nào không?
- Nhìn một hệ thống, bạn đề xuất được loại API nào dùng cho nó?

Bài tập về nhà & chuẩn bị cho buổi 2

BÀI TẬP VỀ NHÀ

Deadline: trước buổi học tiếp theo · nộp qua LMS

1 Tìm 3 API public thật viết về một domain quan tâm

Gợi ý: GitHub REST, OpenWeather, ExchangeRate-API, Spotify Web API, public-apis repo trên GitHub. Mỗi API trả lời: *loại nào (REST/GraphQL/gRPC)? base URL? auth method? có versioning? mỗi resource identifier là số hay UUID?*

2 Đọc Higginbotham Ch.1 + Geewax Ch.1

Tập trung vào: định nghĩa API, resource, và 5 đặc điểm của một API tốt. So sánh với 5 đặc điểm đã học trong slide 05 — có điểm nào Geewax/Higginbotham bổ sung?

3 Hoàn thiện bài 6 + mở rộng

Sau khi CRUD chạy được, thêm: (a) tìm kiếm `GET /books?q=...`; (b) sort theo `?sort=title`; (c) bắt buộc field `year` là số ≥ 1900 .

Số lượng: 3 API + 5 câu so sánh + code mở rộng. Nộp 1 file PDF

BUỔI 2 SẮP TỚI

REST Architecture & HTTP Fundamentals

Sẽ bao gồm

- 6 nguyên tắc REST chi tiết
- HTTP methods: GET/POST/PUT/PATCH/DELETE
- Status codes đầy đủ hơn slide 23
- HTTP headers quan trọng (Content-Type, Accept, Authorization)
- Content negotiation (JSON/XML/YAML)
- Mở rộng bài 6 thành RESTful

Đọc trước

Geewax Ch.1

Câu hỏi cho buổi 2

"PUT vs PATCH khác nhau thế nào?"

"Khi nào 404 khác 410 Gone?"

"Có bao nhiêu cache-control directive?"